```
    birth = models.DateField()
    description = models.TextField(blank=True)
    picture = models.ImageField(blank=True, null=True)

    ...
```

A new description field for the musician; it uses blank=True, so its content can be an empty string.

To store a picture you use an ImageField. As the field is optional, it can be both blank and NULL.

With the model in place, a new view is required, so the user can edit it. This is shown in the following listing.

Listing E.25 A view for editing a `Musician` object

```
# RiffMates/bands/views.py
...
def edit_musician(request, musician_id=0):
    if musician_id != 0:
        musician = get_object_or_404(Musician, id=musician_id)
        if (
            not request.user.is_staff
            and not request.user.userprofile.musician_profiles.filter(
                id=musician_id
            ).exists()
        ):
            raise Http404("Can only edit controlled musicians")

    if request.method == "GET":
        if musician_id == 0:
            form = MusicianForm()
        else:
            form = MusicianForm(instance=musician)

    else:  # POST
        if musician_id == 0:
            musician = Musician.objects.create(birth=date(1900, 1, 1))

        form = MusicianForm(request.POST, request.FILES,
            instance=musician)

        if form.is_valid():
            musician = form.save()

            # Add the musician to the user's profile if this isn't staff
            if not request.user.is_staff:
                request.user.userprofile.musician_profiles.add(musician)

            return redirect("musicians")

    # Was a GET, or Form was not valid
    data = {
        "form": form,
    }

    return render(request, "edit_musician.html", data)
```

The view takes a musician's ID as an argument, using 0 to indicate a new musician is to be created.

For a nonzero ID, fetch the corresponding Musician object.

Ensure only a staff member or someone associated with the Musician object can edit it.

GET is called to present the form.

For new objects, use an empty form; for editing an existing object, prepopulate the form with the musician's info.

POST is called when the form is submitted.

For a new entry, start with a default Musician object.

Validate the form's contents.

Populate the form with the contents of the POST, any uploaded files, and the existing Musician object.

Saving the form writes the Musician object.

After a successful edit, send the user to the Musician Listing page.

If the user is not staff, associate their profile with the Musician object; this can be called multiple times while creating a single connection.

For a GET or failed POST, present the form to the user; the context data contains the form object, which, for a failed POST, contains the previously submitted data.